

# Boot Chain and Image Format

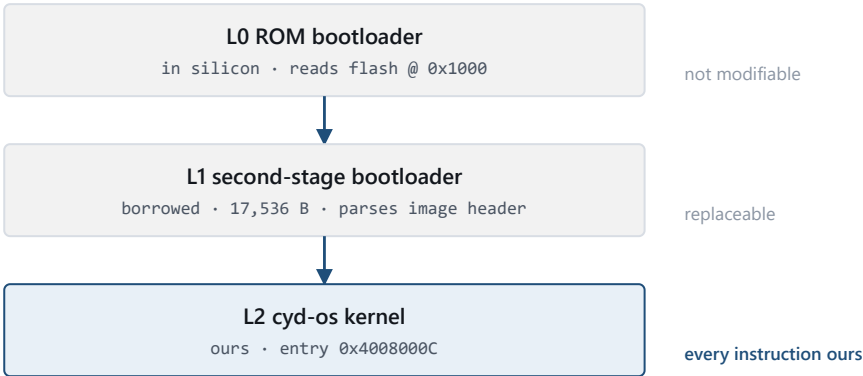
UM-CYDOS-002

| DOCUMENT     | REVISION         | STATUS                         | CLASSIFICATION |
|--------------|------------------|--------------------------------|----------------|
| UM-CYDOS-002 | 1.0 · 2026-08-14 | current — measured on hardware | Internal       |

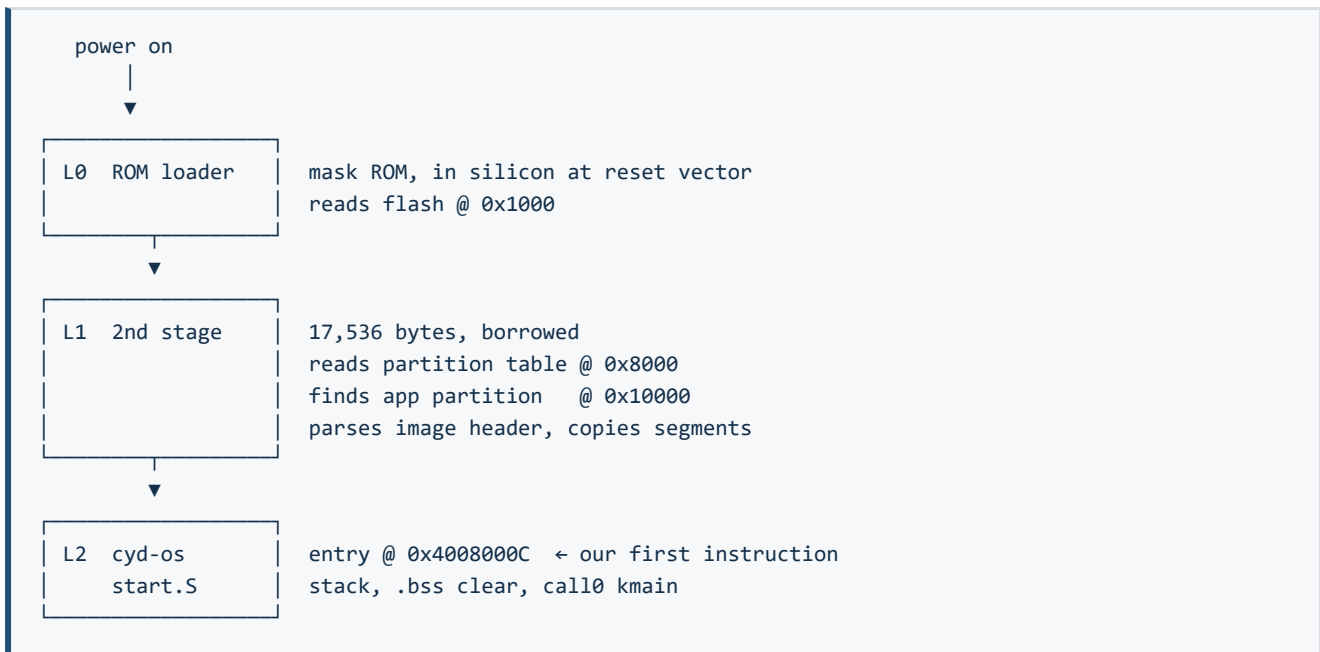
## 1. Abstract

This report documents the complete path from power-on to the first instruction of cyd-os, the binary format the kernel image must satisfy, and the measured boot trace from the target board. It answers the question "how does this C code actually run on the device" precisely enough to debug a failure to boot.

## 2. The four stages



**Figure 1.** The four boot stages. Only L2 upward is project code; the interface to L1 is the image header alone.



## 2.1 Stage L0 — mask ROM

Executes from ROM on reset. Determines boot mode from strapping pins (GPIO0 high = normal flash boot), configures the SPI flash interface, and loads the second-stage bootloader from flash offset `0x1000`. It also brings up UART0 at 115200 baud for its own diagnostic output — a side effect cyd-os currently depends on (see §6).

Not modifiable.

## 2.2 Stage L1 — second-stage bootloader

Borrowed from the CYD PlatformIO project. Responsibilities relevant to us:

1. Read the partition table at `0x8000`.
2. Locate the application partition (offset `0x10000`).
3. Read the image header at that offset.
4. Copy each declared segment to its declared load address.
5. Verify the image checksum and SHA-256 hash.
6. Jump to the declared entry point.

**The interface between L1 and cyd-os is entirely the image header.** Nothing else is shared — no symbols, no runtime, no calling convention. This is why the bootloader is replaceable without touching kernel code.

## 2.3 Stage L2 — cyd-os

Begins at `_start` in `kernel/start.S`. See UM-CYDOS-003 for why this code can assume a call0 environment.

# 3. Image format

`esptool1 elf2image` converts the linked ELF into the format L1 expects. The produced header declares, for each segment, a length and a load address, plus one entry point for the whole image.

Measured output for the current build:

```

Image version: 1
Entry point: 0x4008000C
2 segments
  Segment 1: len 0x00188 load 0x3FFB0000 file_offs 0x018 [BYTE_ACCESSIBLE, DRAM]
  Segment 2: len 0x002E0 load 0x40080000 file_offs 0x1A8 [IRAM]
Checksum: 0x8F (valid)
Validation hash: 8e3a272a...5c72f3 (valid)

```

Both load addresses correspond directly to the `MEMORY` regions declared in `kernel/linker.ld`. The entry point is `_start + 0x0C` rather than `+0x00` because the assembler places the literal pool at the head of the section; see UM-CYDOS-005 §4.

**There is no magic in this format.** It is a list of "copy N bytes to address A", followed by "jump to address E". Any producer that emits a conforming header will boot.

## 4. Measured boot trace

Captured 2026-08-14 from the target board on COM5 at 115200 baud, with the serial port opened *before* reset so no output was missed.

```

ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1184
load:0x40078000,len:13232
load:0x40080400,len:3028
entry 0x400805e4

=====
cyd-os milestone 0 – kernel alive
=====

```

### 4.1 Reading the trace

| Line                               | Meaning                                                                                                                                                                     |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ets Jul 29 2019                    | ROM build date — identifies the silicon revision's ROM                                                                                                                      |
| rst:0x1 (POWERON_RESET)            | Clean cold boot. <b>Diagnostically important:</b> a watchdog or panic reset shows a different code here, and is the first thing to check when a kernel change stops booting |
| boot:0x13<br>(SPI_FAST_FLASH_BOOT) | Normal flash boot; GPIO0 was high                                                                                                                                           |
| mode:DIO, clock div:2              | Flash interface configuration                                                                                                                                               |
| load:0x... x3                      | <b>The bootloader loading itself, not cyd-os</b>                                                                                                                            |
| entry 0x400805e4                   | Entry into the <i>bootloader</i> , not our kernel                                                                                                                           |

The three `load:` lines and the `entry` line are frequently misread as the kernel loading. They are not. They are L1 placing its own segments before it has even read our partition. Our own load is silent — L1 does not log it — and the first evidence of cyd-os is the banner.

## 4.2 Verified handoff

The banner appearing at all proves the header was well-formed, the checksum matched, both segments were copied, and the CPU reached `0x4008000C`. The self-check lines that follow prove more; see UM-CYDOS-006.

## 5. Flash layout

| Offset               | Contents                | Size     | Origin   |
|----------------------|-------------------------|----------|----------|
| <code>0x1000</code>  | Second-stage bootloader | 17,536 B | Borrowed |
| <code>0x8000</code>  | Partition table         | 3,072 B  | Borrowed |
| <code>0x10000</code> | cyd-os kernel image     | 1,216 B  | Ours     |

Everything from `0x10000` onward is available to the project. The two borrowed regions total under 21 KB.

## 6. Known dependencies on L1 behaviour

These are places where cyd-os currently relies on the bootloader or ROM having done something, and would break if L1 were replaced naively.

1. **UART0 is already configured.** The kernel writes bytes into the TX FIFO without setting baud rate, pin mux, or line discipline. This works because the ROM configured UART0 at 115200 for its own output. A replacement L1 that did not do this would produce a silent kernel. *Resolution: implement UART configuration in the kernel — small, and removes the dependency.*
2. **Flash cache is configured but unused.** M0 executes entirely from RAM. When the kernel outgrows IRAM it will begin depending on the cache mapping L1 set up, which is currently unexamined.
3. **CPU clock.** Running at whatever L1 selected. No kernel code reads or sets the clock, so all current timing (including the M0 delay loop) is of unknown absolute rate.

## 7. Failure modes and first diagnostic step

| Symptom                                                    | Most likely cause                                                                  |
|------------------------------------------------------------|------------------------------------------------------------------------------------|
| No output at all                                           | Image checksum invalid, or entry point outside a loaded segment                    |
| ROM trace then silence                                     | Kernel reached but faulted before UART output — suspect stack or <code>.bss</code> |
| Boot loop with <code>rst:0x3</code> / <code>rst:0xc</code> | Watchdog — kernel hung before feeding or disabling it                              |
| Garbled output                                             | Baud mismatch — the ROM's rate was changed or the monitor disagrees                |

In every case the **reset reason on the first line** is the highest-value single datum, which is why the capture procedure resets the board explicitly rather than attaching to an already-running system.

## 8. Reproduction

---

```
cd C:\Users\nobod\Projects\cyd-os
.\build.ps1 -Flash -Port COM5
# then capture with the port opened before reset
```

See UM-CYDOS-005 for the capture harness and why attaching after reset loses the banner.

## 9. References

---

- UM-CYDOS-004 — Memory Map (segment addresses and the overlap risk)
- UM-CYDOS-005 — Build and Flash Pipeline
- UM-CYDOS-006 — Milestone 0 Verification Report